

EECE 7205: Fundamentals of Computer Engineering
Assignment #01
Fall 2026

Uriel Olayinka

September 25, 2026

GitHub repository: <https://github.com/uriel-olayinka/EECE7205-Assignment01>

Problem 1: Merge Intervals

1. Problem Understanding

The goal is to merge any overlapping intervals within a given collection and return the new set of non-overlapping intervals. The input is an array of interval pairs $[start, end]$, and the output is a sorted array of the merged intervals. We assume that $start \leq end$ for all intervals and that intervals sharing an exact endpoint are considered overlapping. Important edge cases include empty inputs, single intervals, one interval fully contained inside another, identical intervals, and intervals with touching endpoints.

2. Solution Design

We first sort the intervals by their starting values. By sorting them, any intervals that could potentially overlap are placed adjacent to one another. This allows us to merge them in a single iteration by simply comparing each interval to the most recently merged one.

3. Pseudocode

```
MERGE-INTERVALS(I, n)
    if n == 0
        return empty array

    I_copy = copy of array I
    SORT(I_copy, n) // Sort the copy by the .start attribute

    merged = empty array
    merged[1] = I_copy[1]
    k = 1 // Tracks the current length of the merged array

    for i = 2 to n
        if I_copy[i].start <= merged[k].end // The intervals overlap or touch
            merged[k].end = MAX(merged[k].end, I_copy[i].end)
        else
            k = k + 1
            merged[k] = I_copy[i]

    return merged[1:k]
```

4. Correctness Explanation

Because the intervals are sorted by their start times, every new interval we process is guaranteed to start at or after the current one. If an interval does not overlap with the last merged interval, no subsequent interval will either. Using the max function for the end value ensures that a fully enclosed interval does not accidentally shrink the merged boundary. The less-than-or-equal comparison correctly handles intervals that merely touch at the endpoints. Empty or single inputs bypass the main loop and safely return the correct result.

5. Complexity Analysis

The time complexity is $O(n \log n)$ for the best, average, and worst cases. The sorting step takes $O(n \log n)$ time, which dominates the $O(n)$ time it takes to execute the merge loop. The space complexity is $O(n)$ because we store a sorted copy of the input and allocate space for the merged list.

6. C++ Implementation

The solution is implemented in the function `mergeIntervals`.

7. Testing

```
Choose a problem: 1

=== Problem 1: Merge Intervals ===

Test: Example
  Input:  [[1,3], [2,6], [8,10], [15,18]]
  Output: [[1,6], [8,10], [15,18]]

Test: Unsorted + fully contained interval
  Input:  [[8,10], [1,10], [2,3], [15,18]]
  Output: [[1,10], [15,18]]

Test: Touching endpoints
  Input:  [[1,2], [2,3], [5,6]]
  Output: [[1,3], [5,6]]

Test: Edge case: empty input
  Input:  []
  Output: []

Test: Edge case: single interval
  Input:  [[4,7]]
  Output: [[4,7]]
```

Problem 2: Sort Items

1. Problem Understanding

The objective is to sort an array containing only the values 0, 1, and 2 in place. Two separate solutions must be implemented: a counting-based approach and the Dutch National Flag algorithm, which forbids the use of `std::sort()`. The input is an integer array of these three values, and the output is the same array sorted in ascending order. Edge cases involve empty arrays, single-element arrays, arrays with identical numbers, and arrays that are already sorted or reverse sorted.

2. Solution Design

The counting-based solution iterates through the array to count how many 0s, 1s, and 2s exist, and then overwrites the array with those exact quantities in order. The Dutch National Flag solution uses three pointers to divide the array into specific regions for 0s, 1s, unexamined elements, and 2s. This sorts the array in a single pass by swapping elements directly into their designated regions without counting.

3. Pseudocode

```
COUNTING-SORT-ITEMS(A, n)
    counts = [0, 0, 0]
    for i = 1 to n
        counts[A[i]] = counts[A[i]] + 1

    index = 1
    for item = 0 to 2
        for j = 1 to counts[item]
            A[index] = item
            index = index + 1
```

```
DUTCH-FLAG-SORT(A, n)
    low = 1
    mid = 1
    high = n

    while mid <= high
        if A[mid] == 0
            swap A[low] and A[mid]
            low = low + 1
            mid = mid + 1
        else if A[mid] == 1
            mid = mid + 1
        else
            swap A[mid] and A[high]
            high = high - 1
```

4. Correctness Explanation

The counting method ensures correctness by explicitly tracking and replacing the exact quantities of each required number. The Dutch National Flag method relies on regional boundaries. Swapping a 0 to the low pointer places it correctly, swapping a 2 to the high pointer places it correctly, and 1s are simply left in the middle region. The process finishes exactly when all unexamined items are evaluated. Both methods naturally handle duplicate values and easily bypass execution if the input is empty.

5. Complexity Analysis

For both solutions, the best, average, and worst-case time complexity is $O(n)$. The counting method requires two iterations, and the Dutch National Flag method requires one iteration, performing a fixed amount of work per element regardless of the initial order. The space complexity for both is $O(1)$ because the counting method uses three integer counters and the Dutch National Flag method sorts completely in place.

6. C++ Implementation

The solutions are implemented in the functions `countingSortItems` and `dutchFlagSort`.

7. Testing

```
Choose a problem: 2

=== Problem 2: Sort Items ===

Test: Example
  Input:  [2, 0, 2, 1, 1, 0]
  Output (counting sort): [0, 0, 1, 1, 2, 2]
  Output (Dutch Flag sort): [0, 0, 1, 1, 2, 2]

Test: Already sorted
  Input:  [0, 0, 1, 1, 2, 2]
  Output (counting sort): [0, 0, 1, 1, 2, 2]
  Output (Dutch Flag sort): [0, 0, 1, 1, 2, 2]

Test: Reverse order
  Input:  [2, 2, 1, 1, 0, 0]
  Output (counting sort): [0, 0, 1, 1, 2, 2]
  Output (Dutch Flag sort): [0, 0, 1, 1, 2, 2]

Test: Edge case: all the same value
  Input:  [1, 1, 1]
  Output (counting sort): [1, 1, 1]
  Output (Dutch Flag sort): [1, 1, 1]

Test: Edge case: single element
  Input:  [2]
  Output (counting sort): [2]
  Output (Dutch Flag sort): [2]

Test: Edge case: empty input
  Input:  []
  Output (counting sort): []
  Output (Dutch Flag sort): []
```

Problem 3: Largest Number

1. Problem Understanding

The goal is to arrange a sequence of nonnegative integers such that their concatenation creates the largest possible number, returning the final result as a string. The input is a vector of integers, and the output is a string because the resulting number will likely exceed standard integer limits. Edge cases include arrays composed entirely of zeros, single numbers, inputs with shared prefixes (like 12 and 121), and empty inputs.

2. Solution Design

We implement a custom sorting function based on string concatenation rather than standard numerical sorting. We compare two numbers by joining them into strings in both possible orders. If string $a + b$ is greater than $b + a$, then a must come before b in the sorted array to ensure the maximum possible final value.

3. Pseudocode

```
LARGEST-NUMBER(A, n)
    if n == 0
        return ""

    parts = empty array of size n
    for i = 1 to n
        parts[i] = string version of A[i]

    SORT(parts, n) // Rule: string a comes before b if (a + b) > (b + a)

    if parts[1] == "0"
        return "0"

    result = ""
    for i = 1 to n
        result = result + parts[i]

    return result
```

4. Correctness Explanation

The custom sorting rule guarantees that swapping any adjacent components in the final constructed string would only decrease its total value. Comparing the two possible concatenated sequences automatically resolves issues with numbers of varying lengths and shared prefixes. If the very first sorted piece is a "0", the highest possible number is 0, so we return a single "0" rather than a string of multiple zeros. Duplicates compare equally and function correctly regardless of order, and empty inputs return an empty string safely.

5. Complexity Analysis

The time complexity is $O(n \log n \cdot d)$ in all cases, where n is the total number of integers and d is the maximum digit length of those integers. Standard sorting algorithms perform $O(n \log n)$ comparisons, and each comparison requires evaluating strings of length $O(d)$. The space complexity is $O(n \cdot d)$ to accommodate the string conversions of the numbers and the final concatenated result string.

6. C++ Implementation

The solution is implemented in the function `largestNumber`.

7. Testing

```
Choose a problem: 3
```

```
=== Problem 3: Largest Number ===
```

```
Test: Example
```

```
Input:  [3, 30, 34, 5, 9]
```

```
Output: "9534330"
```

```
Test: Simple pair
```

```
Input:  [10, 2]
```

```
Output: "210"
```

```
Test: Shared prefix
```

```
Input:  [121, 12]
```

```
Output: "12121"
```

```
Test: Edge case: all zeros
```

```
Input:  [0, 0, 0]
```

```
Output: "0"
```

```
Test: Edge case: single number
```

```
Input:  [7]
```

```
Output: "7"
```

```
Test: Edge case: empty input
```

```
Input:  []
```

```
Output: ""
```

Problem 4: Group Anagrams

1. Problem Understanding

The objective is to group together words that are anagrams of one another, maintaining the original input sequence of the words within each group. The input is a vector of strings, and the output is a vector of grouped string arrays. The matching is case-sensitive, and duplicate words must be retained in their respective groups. Edge cases include empty inputs, single words, empty strings, and words that do not have any anagram partners.

2. Solution Design

We process the input using a hash map where the sorted letters of a word act as the dictionary key. Because anagrams consist of the exact same characters, sorting them yields the identical key string. As we iterate through the input list, we sort each word, find its matching key in the hash map, and append the original word to that specific group.

3. Pseudocode

```
GROUP-ANAGRAMS(A, n)
    group_index = empty hash map
    groups = empty array of lists
    k = 0 // Tracks the number of unique groups

    for i = 1 to n
        word = A[i]
        key = word with characters sorted

        if key is not in group_index
            k = k + 1
            group_index[key] = k
            groups[k] = empty list

        idx = group_index[key]
        add word to the end of groups[idx]

    return groups[1:k]
```

4. Correctness Explanation

By definition, words share the same exact characters and frequencies if they are anagrams, which means sorting them guarantees they map to the exact same hash map key. Because we process the input array from start to finish and append matches to the end of the lists, the groups inherently preserve the original input order. Identical duplicates generate the same key and are placed side-by-side. An empty string sorts to an empty string, forming a valid unique group, while words without anagrams simply form a group of one.

5. Complexity Analysis

The time complexity is $O(n \cdot k \log k)$ across best, average, and worst cases. Here, n represents the total number of words and k represents the maximum length of a word. Hashing lookup takes $O(k)$ on average, but sorting the characters of each individual word dictates the overarching $O(k \log k)$ multiplier. Space complexity is $O(n \cdot k)$ to retain the sorted keys in the hash map and generate the final grouped array structure.

6. C++ Implementation

The solution is implemented in the function `groupAnagrams`.

7. Testing

```
Choose a problem: 4

=== Problem 4: Group Anagrams ===

Test: Example
  Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
  Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

Test: Single group
  Input:  ["abc", "cab", "bca"]
  Output: [["abc", "cab", "bca"]]

Test: No anagrams
  Input:  ["dog", "cat", "fish"]
  Output: [["dog"], ["cat"], ["fish"]]

Test: Edge case: empty string
  Input:  [""]
  Output: [[""]]

Test: Edge case: single word
  Input:  ["a"]
  Output: [["a"]]

Test: Edge case: empty input
  Input:  []
  Output: []
```

Problem 5: Meetings

1. Problem Understanding

The goal here is to determine if a single person can attend a list of scheduled meetings without any time conflicts. We take an array of meeting intervals and return true if there are no overlaps, or false otherwise. We assume that one meeting can start at the exact same time another ends without causing a conflict. Important edge cases to consider are empty inputs, single meetings, back-to-back schedules, and unsorted inputs.

2. Solution Design

We can solve this by first sorting the meetings by their start times. After sorting, we need to iterate through the list and compare each meeting to the one immediately preceding it. If any meeting begins strictly before the previous one finishes, we know there is an overlap and can stop checking.

3. Pseudocode

```
CAN-ATTEND-ALL-MEETINGS(meetings, n)
    if n < 2
        return true

    meetings_copy = copy of array meetings
    SORT(meetings_copy, n) // Sort the copy by the .start attribute

    for i = 2 to n
        if meetings_copy[i].start < meetings_copy[i-1].end // Strictly less means they
        overlap
            return false

    return true
```

4. Correctness Explanation

By sorting the meetings first, we guarantee that every meeting starts after or at the same time as the previous one. This means we only need to check adjacent pairs to find overlaps. The strict less-than operator ensures that back-to-back meetings are allowed to happen. Edge cases like identical meetings or a meeting inside another will trigger a conflict, while empty or single inputs safely skip the check and return true.

5. Complexity Analysis

The time complexity is $O(n \log n)$ across all cases because sorting the array takes $O(n \log n)$ time and dominates the $O(n)$ scan. The space complexity is $O(n)$ because we create a sorted copy of the input array so the original data remains unmodified.

6. C++ Implementation

The solution is implemented in the function `canAttendAllMeetings`.

7. Testing

```
Choose a problem: 5

=== Problem 5: Meetings ===
Example (overlap): [[0,30], [5,10], [15,20]] -> false
No overlap, unsorted: [[7,10], [2,4]] -> true
Back-to-back meetings: [[1,5], [5,8], [8,9]] -> true
Meeting inside a longer one: [[1,10], [2,3], [4,5]] -> false
Edge case: empty input: [] -> true
Edge case: single meeting: [[3,6]] -> true
```

Problem 6: K Closest Points to the Origin

1. Problem Understanding

The objective is to find the k closest points to the origin from a given set of coordinates. We take a list of points and an integer k , and output a list of the k closest points. We assume standard Euclidean distance and don't need a specific tie-breaking order for points at the exact same distance. Edge cases include requesting 0 points, requesting more points than available, and multiple points sharing the exact same distance.

2. Solution Design

We avoid floating-point math by comparing the squared distances instead of true distances. The sorting solution simply sorts all points by this distance and picks the first k elements. The more efficient heap solution maintains a max-heap of size k , adding points and removing the farthest one whenever the size exceeds k .

3. Pseudocode

```
SQUARED-DISTANCE(p)
    return p.x * p.x + p.y * p.y

K-CLOSEST-BY-SORTING(P, n, k)
    if k <= 0
        return empty array

    P_copy = copy of array P
    SORT(P_copy, n) // Sort by SQUARED-DISTANCE in ascending order

    limit = min(k, n)
    return P_copy[1:limit]
```

```
K-CLOSEST-BY-HEAP(P, n, k)
    if k <= 0
        return empty array

    heap = empty max-heap // Ordered by SQUARED-DISTANCE, largest at top

    for i = 1 to n
        add P[i] to heap
        if size of heap > k
            remove top element from heap

    limit = size of heap
    result = empty array of size limit

    for i = limit downto 1
        result[i] = remove top element from heap
```

```
return result
```

4. Correctness Explanation

The sorting method naturally puts the closest points at the front of the array. The heap method works by always keeping the closest points seen so far, discarding the farthest candidate whenever the limit is exceeded. Both approaches handle identical points and ties correctly, and if k is larger than the input size, all points are returned safely. An empty list is correctly returned if k is zero or the input is empty.

5. Complexity Analysis

The sorting solution has a time complexity of $O(n \log n)$ and a space complexity of $O(n)$. The heap solution has a time complexity of $O(n \log k)$ since we insert n elements into a heap of size k , and requires $O(k)$ space. Best, average, and worst cases remain the same for both methods because they do not depend on the input order.

6. C++ Implementation

The solutions are implemented in the functions `kClosestBySorting` and `kClosestByHeap`.

7. Testing

```
Choose a problem: 6

=== Problem 6: K Closest Points ===

Test: Example
Input: [(1,3), (-2,2), (5,8), (0,1)], k = 2
Output (sorting): [(0,1), (-2,2)]
Output (heap): [(0,1), (-2,2)]

Test: All points same distance
Input: [(1,1), (-1,-1), (1,-1), (-1,1)], k = 3
Output (sorting): [(1,1), (-1,-1), (1,-1)]
Output (heap): [(1,1), (-1,-1), (1,-1)]

Test: Edge case: k = 0
Input: [(1,2), (3,4)], k = 0
Output (sorting): []
Output (heap): []

Test: Edge case: k = number of points
Input: [(2,3), (4,5)], k = 2
Output (sorting): [(2,3), (4,5)]
Output (heap): [(2,3), (4,5)]

Test: Edge case: k larger than number of points
Input: [(1,2), (3,4)], k = 5
Output (sorting): [(1,2), (3,4)]
Output (heap): [(1,2), (3,4)]

Test: Edge case: single point
Input: [(7,8)], k = 1
Output (sorting): [(7,8)]
Output (heap): [(7,8)]

Test: Edge case: empty input
Input: [], k = 3
Output (sorting): []
Output (heap): []
```

Problem 7: Top K Frequent Elements

1. Problem Understanding

The goal is to find the k most frequently occurring numbers in an array. We need to return a list of k values ordered by frequency, breaking any ties by putting the smaller numerical value first. The input can have negative numbers and duplicates. Edge cases involve empty arrays, ties in frequency, single elements, and requesting more values than the number of unique elements.

2. Solution Design

We can solve this by first counting the frequency of each number using a hash map. Next, we sort the distinct values based on their counts in descending order. If two counts are the same, we sort them by their actual value in ascending order. Finally, we return the first k elements from this sorted list.

3. Pseudocode

```
TOP-K-FREQUENT(A, n, k)
    counts = empty hash map

    for i = 1 to n
        value = A[i]
        if value is not in counts
            counts[value] = 0
        counts[value] = counts[value] + 1

    entries = empty array
    m = 0 // Tracks the number of unique elements

    for each unique key in counts
        m = m + 1
        entries[m] = (key, counts[key])

    SORT(entries, m) // Rule: higher count first; if equal, smaller key first

    limit = min(k, m)
    result = empty array of size limit

    for i = 1 to limit
        result[i] = entries[i].key

    return result
```

4. Correctness Explanation

The hash map accurately tracks how many times each number appears. The custom sorting rule specifically guarantees that we sort by highest frequency first and break ties using the smaller

number. Negative numbers are treated naturally as smaller values. If k is larger than the number of unique elements, the loop just returns all of them.

5. Complexity Analysis

The time complexity is $O(n + m \log m)$, where n is the total elements and m is the unique elements. In the best case where all values are identical, the time complexity becomes $O(n)$. In the worst case where all values are unique, this becomes $O(n \log n)$. The space complexity is $O(m)$ to store the hash map and the unique entries.

6. C++ Implementation

The solution is implemented in the function `topKFrequent`.

7. Testing

```
Choose a problem: 7

=== Problem 7: Top K Frequent Elements ===

Test: Example
  Input:  [1, 1, 1, 2, 2, 3], k = 2
  Output: [1, 2]

Test: Tie: smaller value first
  Input:  [5, 5, 3, 3, 1], k = 2
  Output: [3, 5]

Test: Negative numbers
  Input:  [-1, -1, 2, 2, 2, -3], k = 2
  Output: [2, -1]

Test: Edge case: single element
  Input:  [1], k = 1
  Output: [1]

Test: Edge case: k larger than distinct values
  Input:  [4, 4, 6], k = 5
  Output: [4, 6]

Test: Edge case: empty input
  Input:  [], k = 2
  Output: []
```

Problem 8: Merge K Sorted Arrays

1. Problem Understanding

The objective is to combine k individually sorted arrays into one large sorted array without having to sort the final result from scratch. The input is a list of k sorted arrays, and the output is a single combined array. We assume all input arrays are already sorted and we need to keep all duplicate values. Edge cases include empty arrays, arrays of different lengths, and having no arrays at all.

2. Solution Design

The best approach is using a min-heap to keep track of the smallest current element from each array. We add the first element of each array to the heap. We then continuously remove the smallest element to add to our final result, replacing it in the heap with the next element from that same array.

3. Pseudocode

```
MERGE-K-SORTED-ARRAYS(A, k)
    min_heap = empty min-heap // Ordered by value. Stores tuples: (value, array_index,
    element_index)

    for i = 1 to k
        if length of A[i] > 0
            add (A[i][1], i, 1) to min_heap

    merged = empty array
    m = 0 // Tracks the number of elements in the merged array

    while min_heap is not empty
        (val, arr_idx, elem_idx) = remove top element from min_heap

        m = m + 1
        merged[m] = val

        if elem_idx < length of A[arr_idx]
            next_idx = elem_idx + 1
            add (A[arr_idx][next_idx], arr_idx, next_idx) to min_heap

    return merged[1:m]
```

4. Correctness Explanation

Because each individual array is sorted, the smallest remaining number overall has to be one of the current front elements. The min-heap always gives us the smallest of these candidates. By replacing the extracted element with the next one from its source array, we guarantee the final

output is built in sorted order. Empty arrays are simply skipped, and equal values are processed in any valid order.

5. Complexity Analysis

The time complexity is $O(N \log k)$, where N is the total number of elements, because each element is processed in a heap of size k . This applies to best, average, and worst cases. The space complexity is $O(k)$ for the heap, plus the $O(N)$ space needed for the merged result.

6. C++ Implementation

The solution is implemented in the function `mergeKSortedArrays`.

7. Testing

```
Choose a problem: 8

=== Problem 8: Merge K Sorted Arrays ===

Test: Example
  Input:  [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
  Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Test: Different lengths, duplicates
  Input:  [[1, 3, 3], [2, 3], [0, 10]]
  Output: [0, 1, 2, 3, 3, 3, 10]

Test: Edge case: one array is empty
  Input:  [[1, 3], [], [2, 2, 4]]
  Output: [1, 2, 2, 3, 4]

Test: Edge case: single array
  Input:  [[5, 6, 7]]
  Output: [5, 6, 7]

Test: Edge case: no arrays
  Input:  []
  Output: []

Test: Edge case: arrays with empty arrays
  Input:  [[], [], []]
  Output: []
```

Problem 9: Kth Largest Element

1. Problem Understanding

The goal is to find the k th-largest number in an unsorted array without having to completely sort the entire array. The input is an array and a target integer k , and we output the value at that k th-largest position. Duplicates are counted as separate positions. Edge cases include requesting the maximum or minimum value, invalid k values, and arrays with duplicate or negative numbers.

2. Solution Design

We can find the answer efficiently by maintaining a min-heap that holds exactly k elements. As we iterate through the array, we add each number to the heap and remove the smallest one whenever the size exceeds k . By the end of the loop, the top of the heap will be the k th-largest element.

3. Pseudocode

```
FIND-KTH-LARGEST(A, n, k)
    if k < 1 or k > n
        return "Invalid k"

    min_heap = empty min-heap

    for i = 1 to n
        add A[i] to min_heap

        if size of min_heap > k
            remove top element from min_heap

    return top element of min_heap
```

4. Correctness Explanation

The min-heap keeps track of the k largest numbers we have seen so far. Any time we remove a number, we are discarding the smallest of a $k + 1$ group, meaning it cannot be part of the top k . When the loop finishes, the smallest value remaining in the heap is exactly the k th-largest overall. Duplicates are treated as individual entries, and invalid values of k are caught before the heap is even used.

5. Complexity Analysis

The time complexity is $O(n \log k)$ for all cases since every element is pushed into a heap of size k . The space complexity is $O(k)$ to maintain the min-heap.

6. C++ Implementation

The solution is implemented in the function `findKthLargest`.

7. Testing

```
Choose a problem: 9

=== Problem 9: Kth Largest Element ===

Test: Example
  Input:  [3, 2, 1, 5, 6, 4], k = 2
  Output: 5

Test: Single element
  Input:  [10], k = 1
  Output: 10

Test: All elements same
  Input:  [7, 7, 7, 7], k = 3
  Output: 7

Test: Negative numbers
  Input:  [-1, -2, -3, -4], k = 2
  Output: -2

Test: Edge case: k larger than array size
  Input:  [1, 2], k = 5
  Output: Invalid k

Test: Edge case: k = 0
  Input:  [1, 2, 3], k = 0
  Output: Invalid k

Test: Duplicates count separately
  Input:  [3, 2, 3, 1, 2, 4, 5, 5, 6], k = 4
  Output: 4
```

Problem 10: Count Inversions

1. Problem Understanding

The goal is to count how many pairs of numbers in an array are out of order, meaning the earlier number is strictly greater than the later number. We take an integer array and output the total count of these inversions. We assume the original array should not be modified and equal values do not count as inversions. Important edge cases include already sorted arrays, reverse sorted arrays, and duplicate elements.

2. Solution Design

We use a modified Merge Sort to count the inversions efficiently. We recursively split the array in half to count inversions within the left and right sections. During the merge step, whenever we pull an element from the right half before elements in the left half, we know it forms an inversion with all remaining left-half elements. We use a 64-bit integer to store the count to prevent overflow.

3. Pseudocode

```
COUNT-INVERSIONS(A, n)
    if n < 2
        return 0

    temp = empty array of size n
    A_copy = copy of array A
    return SORT-AND-COUNT(A_copy, temp, 1, n)
```

```
SORT-AND-COUNT(A, temp, left, right)
    if left >= right
        return 0

    mid = floor((left + right) / 2)

    count = SORT-AND-COUNT(A, temp, left, mid)
    count = count + SORT-AND-COUNT(A, temp, mid + 1, right)
    count = count + MERGE-AND-COUNT(A, temp, left, mid, right)

    return count
```

```
MERGE-AND-COUNT(A, temp, left, mid, right)
    i = left
    j = mid + 1
    k = left
    count = 0

    while i <= mid and j <= right
        if A[i] <= A[j]
            temp[k] = A[i]
            i = i + 1
        else
            count = count + (mid - i + 1)
            temp[k] = A[j]
            j = j + 1
        k = k + 1

    while i <= mid
        temp[k] = A[i]
        i = i + 1
        k = k + 1

    while j <= right
        temp[k] = A[j]
        j = j + 1
        k = k + 1

    return count
```

```

        else
            temp[k] = A[j]
            j = j + 1
            count = count + (mid - i + 1) // Remaining left elements form inversions
            k = k + 1

// Copy any remaining elements from the left half
while i <= mid
    temp[k] = A[i]
    i = i + 1
    k = k + 1

// Copy any remaining elements from the right half
while j <= right
    temp[k] = A[j]
    j = j + 1
    k = k + 1

// Copy the merged sorted elements back into the working array
for x = left to right
    A[x] = temp[x]

return count

```

4. Correctness Explanation

Every inversion is either fully in the left half, fully in the right half, or split between them. The recursive steps count the internal inversions, while the merge step accurately counts the split pairs. Because both halves are sorted during the merge, placing a right-half element means it is smaller than every left-half element still waiting to be merged. Equal values default to taking the left element, ensuring no false inversions are counted.

5. Complexity Analysis

The time complexity is $O(n \log n)$ in all cases, identical to standard Merge Sort, because the array is divided logarithmically and merged in linear time. The space complexity is $O(n)$ for the temporary merge array and $O(\log n)$ for the recursion stack.

6. C++ Implementation

The solution is implemented in the functions `countInversions`, `sortAndCount`, and `mergeAndCount`.

7. Testing

```
Choose a problem: 10

=== Problem 10: Count Inversions ===

Test: Example
  Input:  [2, 4, 1, 3, 5]
  Output: 3 (optimized), 3 (brute force)
  [match]

Test: Already sorted
  Input:  [1, 2, 3, 4, 5]
  Output: 0 (optimized), 0 (brute force)
  [match]

Test: Reverse order
  Input:  [5, 4, 3, 2, 1]
  Output: 10 (optimized), 10 (brute force)
  [match]

Test: Duplicates
  Input:  [1, 3, 2, 3, 1]
  Output: 4 (optimized), 4 (brute force)
  [match]

Test: Edge case: single element
  Input:  [42]
  Output: 0 (optimized), 0 (brute force)
  [match]

Test: Edge case: empty input
  Input:  []
  Output: 0 (optimized), 0 (brute force)
  [match]

Test: Edge case: all same elements
  Input:  [7, 7, 7, 7]
  Output: 0 (optimized), 0 (brute force)
  [match]
```

AI Disclosure

AI was used to generate a comprehensive LaTeX template for the writeup. It was also used to create a clean C++ implementation for running test cases (`runTest`), not the test cases themselves. AI was used to debug various coding and compilation errors.